Prompt methodology

The output of the LLM is depended by the prompt it gets. Therefore, it is important to create the prompt carefully and see the behaviour of the LLM. For the application of LLMs in knowledge graphs that discuss software traceability is this even more the important because the LLM has not been explicitly trained on these tasks. By creating the prompt carefully, LLMs are capable to create desired outputs for tasks they haven't been trained on as well [TraceLLM: Leveraging Large Language Models with Prompt Engineering for Enhanced Requirements Traceability" (Alturayeif et al., 2026)]

Since the LLM will work in a continuous environment, the reasoning feature that comes with many state-of-the-art models will be minimized. The reason for this is that the reasoning feature takes more time to generate, this drops the performance and violates the continuous element given to this research.

Both LLMs perform the same task, but have different characteristics on what they prefer, what they are capable of, and how they perform best. For that reason, one prompt will be created to use for both LLMs. This causes that both LLMs will get the same input and none of them will create a better output because it can follow instructions better than the other.

Therefore, I will first list the best practices for each LLM. Then, the shared best practices are used in the prompt. After that, the outcome will be tested and the results will be shown. Depending on the outcome, an improved version of the prompt will be created.

Best practices GPT-5.1

The first insights of creating the best prompt for GPT-5.1 are the following (https://developers.openai.com/cookbook/examples/gpt-5/gpt-5-1_prompting_guide):

- It can help to emphasize via prompting the importance of persistence and completeness. This is because consumption of tokens can come at a cost of not giving a complete answer.
- GPT-5 can be verbose, so be explicit in the instructions as well as the output that should be generated by the LLM.
- Do not write conflicting instructions and be clear in what the LLM is expected to do.

Setting the reasoning model at 'none' improves the efficiency of the model. However, finding similarity between nodes in a part of the graph with interlinked nodes requires the model to think about their choices. Therefore, I will put the reasoning mode to minimal. This lets the model think about their thoughts and makes it able to connect nodes that are not directly related but in between other nodes.

Other best practices follow from the best practices of the GPT-4.1 model. These are (https://developers.openai.com/cookbook/examples/gpt4-1_prompting_guide):

- Allows prompting long contexts up to 1 million tokens
- Prompt organization is important when inserting long prompts. When long prompts are used, consider putting the instructions at the beginning and end of the prompt ideally.
- The structure of the prompt
    - Determine a role and objectives at the start of the prompt

    - Define which reasoning steps the model should make
    - Show what the format of the output should be
    - Show examples
    - Give context
    - final/repeat instructions
- Use delimiters to show sections in the prompt (Markdown, XML, JSON)

Best practices Qwen3.5 4B

Qwen has the following best practices for prompt engineering (https://qwen3lm.com/prompt/)

- Qwen3.5 is build using four building blocks in which the order matters
    - System message
    - User message
    - Assistant message
    - Tool / function call
- Make use of the following structure
    - Role
    - Goal
    - Constraints
    - Output format
- Be specific and clear (don't write conflicting instructions)
- Few shot examples
- Let the model think (chain of thought: 'let's think step by step')
- Ask for structured output

Prompt v1

The first important aspect to notice is that more information about prompt engineering can be found about GPT than Qwen. This might also be the case because GPT is a more advanced model and thus has more capabilities. However, both LLMs also have best practices in common. Both perform better when giving a structure, although Qwen performs better with a certain order. In both cases, being specific, applying delimiters for sections, give examples of the output, and what the output should be, are in common. The prompt will therefore contain the following best practices:

- Be specific regarding the instructions
- Use delimiters
- Use an example of the output
- Let the model think
- Restrict the amount of output tokens (adapt this to the amount of thinking)
- Ask a specific type of output
- Use the following structure
    - Role of the LLM to generate the output
    - Goal of the prompt
    - Give instructions about the steps the model should make
    - Examples of the output
    - Constraints (in my case the schema that the output should adhere to)

- Context (in my case the nodes and edges from the graph)
- Briefly repeat the instructions, goal and output.

The reason for repeating the instructions at both the beginning and end of the prompt is because the prompt will get very long and LLMs might neglect information that is located in the middle or beginning of the prompt (chat link). Also, I chose to use the building blocks for Qwen3.5. The reason for this is that, using building blocks, Qwen is less likely to forget what the model is supposed to adhere to. Since GPT also uses messages as parameter the same way, both LLMs are equal how the information comes in. Furthermore, I will still use the delimiter in both cases. That will enhance the specificity.

Initial prompt

Considering these best practices, the initial prompt is created as:

System message:

Role
You are a knowledge graph engineer specialised in software traceability. Your task is to analyse existing graph nodes and edges and propose new or improved edges that strengthen semantic links between software artefacts.

Constraints
```
{
 "nodes": {
  "Release": {
   "version": { "type": "string" }
  },
  "Issue": {
   "title": { "type": "string" },
   "type": { "type": "string" },
   "status": { "type": "string"},
   "description": { "type": "string" },
   "created": { "type": "date" },
   "resolved": { "type": "date" }
  },
  "Issue:Feature": {
   "unit of work": { "type": "integer" },
   "business value": { "type": "integer" }
  },
  "Issue:Bug": {
   "detected_date": { "type": "date" },
   "root_cause": { "type": "string" },
   "solved": { "type": "string"}
  },
  "Commit": {
   "title": { "type": "string" },
   "message": { "type": "string" },
   "timestamp": { "type": "string" }
  },
  "Developer": {
   "name": { "type": "string" },
   "email": { "type": "string" }
```

```
  },
  "Code": {
    "file_path": { "type": "string" },
    "deleted": { "type": "string" }
  }
 },
 "edges": {
  "issue_included_in_release": {
    "source_id": "Issue",
    "target_id": "Release",
    "label": "included in"
  },
  "issue_updates_issue": {
    "source_id": "Issue",
    "target_id": "Issue",
    "label": "updates"
  },
  "feature_inherits_issue": {
    "source_id": "Feature",
    "target_id": "Issue",
    "label": "is a"
  },
  "bug_inherits_issue": {
    "source_id": "Bug",
    "target_id": "Issue",
    "label": "is a"
  },
  "feature_relates_to_feature": {
    "source_id": "Feature",
    "target_id": "Feature",
    "label": "relates to"
  },
  "bug_caused_by_feature": {
    "source_id": "Bug",
    "target_id": "Feature",
    "label": "caused by"
  },
  "commit_implements_feature": {
    "source_id": "Commit",
    "target_id": "Feature",
    "label": "implements"
  },
  "commit_solves_bug": {
    "source_id": "Commit",
    "target_id": "Bug",
    "label": "solves"
  },
  "developer_creates_commit": {
    "source_id": "Developer",
    "target_id": "Commit",
    "label": "creates"
  },
```

```
  "issue_assigned_to_developer": {
    "source_id": "Issue",
    "target_id": "Developer",
    "label": "assigned to"
  },
  "commit_changes_code": {
    "source_id": "Commit",
    "target_id": "Code",
    "label": "changes"
  }
 }
}
```

User message:

Goal
Given a set of nodes and existing edges from a software traceability knowledge graph, identify missing edges between nodes that are semantically related but not yet connected, and propose synonym edges that replace or supplement vague relationship labels with more precise ones. Return only the new or improved edges in the specified JSON schema with a confidence score higher than 0.85.

Steps
Follow these steps — think through each before writing output:
<step id="1">
Read all nodes inside <graph_content> and build a mental model of what each artefact represents (requirement, test, code module, bug, etc.).
</step>
<step id="2">
Inspect the existing edges. Identify: (a) pairs of nodes with NO edge that share an implied semantic relationship, (b) edges whose label is ambiguous or generic (e.g. "related_to", "links") and could be replaced with a domain-specific synonym.
</step>
<step id="3">
For every candidate edge, ask yourself: (a) Does this relationship add traceability value (e.g. satisfies, implements, validates, depends_on, derived_from, refines)? (b) Is the direction of the edge correct? (c) Is there enough evidence in the node metadata to justify the edge? Only proceed with this edge if your confidence is higher than 0.85.
</step>
<step id="4">
Compose the output. For each new or improved edge, fill every field of the JSON schema. Add your confidence of adding that relationship and a short "explanation" string (max 20 words) that explains why that edge was created.
</step>

Example output
```
{
 "new_edges": [
  {
   "source_id":  " Commit-017",
   "target_id":  " Feature-042",
```

```
    "label":     "Implements ",
    "confidence": 0.91,
    "system":      "LLM",
    "explanation":  "Commit 17 implemented the exact API operations specified in feature 42. Since both have a created date without much time in between, the commit and feature seem to be created for the same purpose but were not linked."
   },
   {
    "source_id":  "Commit-008",
    "target_id":  "File-042",
    "label":     "Renames",
    "confidence": 0.96,
    "system":      "LLM",
    "explanation":  "Commit 8 renames the file rather than changing it. Since the word renaming is more descriptive than changing, I used the word ‘renaming’ as synonym label for this edge.”
."
   }
 ]
}

# Graph content
{
 "sliding_window_events": {
  "nodes": {
   "commit_901": {
    "type": "Commit",
    "title": "Fix auth timeout bug",
    "message": "Adjusted JWT verification token expiration buffer to resolve premature session termination.",
    "timestamp": "2026-05-23T01:15:00Z"
   },
   "code_401": {
    "type": "Code",
    "file_path": "src/auth/middleware.ts",
    "deleted": "false"
   }
  },
  "edges": {
   "edge_e1": {
    "source_id": "commit_901",
    "target_id": "code_401",
    "label": "changes",
    "system": "git_event_stream",
    "confidence": 100,
    "explanation": "Event log captures commit_901 directly modifying middleware.ts."
   }
  }
 },

 "k_hop_neighbourhood": {
  "nodes": {
   "bug_302": {
```

```
      "type": "Issue:Bug",
      "title": "Intermittent 401 on valid user session",
      "status": "in progress",
      "description": "Users are reporting sudden logouts while actively browsing dashboards.",
      "created": "2026-05-22",
      "resolved": null,
      "detected_date": "2026-05-22",
      "root_cause": "Race condition in validation timestamp check.",
      "solved": "no"
    },
    "dev_105": {
      "type": "Developer",
      "name": "Alex Mercer",
      "email": "alex.mercer@company.com"
    },
    "feature_044": {
      "type": "Issue:Feature",
      "title": "Implement RBAC Auth Middleware",
      "status": "done",
      "description": "Establish baseline token validation middleware for role-based access
control.",
      "created": "2026-04-10",
      "resolved": "2026-05-01",
      "unit of work": 5,
      "business value": 80
    }
  },
  "edges": {
    "edge_k1": {
      "source_id": "commit_901",
      "target_id": "bug_302",
      "label": "solves",
      "system": "k_hop_retriever",
      "confidence": 95,
      "explanation": "Hop 1 from sliding window: Commit message explicitly tags or addresses the
context of bug_302."
    },
    "edge_k2": {
      "source_id": "dev_105",
      "target_id": "commit_901",
      "label": "creates",
      "system": "k_hop_retriever",
      "confidence": 100,
      "explanation": "Hop 1 from sliding window: Author metadata of commit_901 resolves to
dev_105."
    },
    "edge_k3": {
      "source_id": "bug_302",
      "target_id": "feature_044",
      "label": "caused by",
      "system": "k_hop_retriever",
      "confidence": 85,
```

```
      "explanation": "Hop 2 from bug_302: Traceability logs show this session regression originated after feature_044 shipped."
    },
    "edge_k4": {
     "source_id": "bug_302",
     "target_id": "dev_105",
     "label": "assigned to",
     "system": "k_hop_retriever",
     "confidence": 100,
     "explanation": "Hop 2 from bug_302: Issue tracker shows dev_105 owns this ticket."
    }
   }
  },

  "vector_similarity_retrieval": {
   "nodes": {
    "bug_101": {
     "type": "Issue:Bug",
     "title": "Token refresh timing anomaly",
     "status": "done",
     "description": "Legacy authorization system occasionally fails when system clocks are out of sync.",
     "created": "2025-11-12",
     "resolved": "2025-11-15",
     "detected_date": "2025-11-12",
     "root_cause": "Hardcoded token validation windows with tight tolerance.",
     "solved": "yes"
    },
    "feature_999": {
     "type": "Issue:Feature",
     "title": "User Session Expiry Notification",
     "status": "planned",
     "description": "Provide UI alerts notifying active users that their authentication token is nearing expiry.",
     "created": "2026-05-19",
     "resolved": null,
     "unit of work": 3,
     "business value": 40
    },
    "code_991": {
     "type": "Code",
     "file_path": "src/security/token_validator.ts",
     "deleted": "false"
    }
   },
   "edges": {}
  }
}
```


Recap

So, you are a knowledge graph engineer for software traceability. Analyse the nodes and edges in graph content, follow steps 1–4, and return ONLY a JSON object adhering the schema above. Propose missing edges and synonym replacements that improve semantic precision.

GPT response


```
{
 "new_edges": [
  {
   "source_id": "commit_901",
   "target_id": "feature_044",
   "label": "implements",
   "confidence": 0.92,
   "system": "LLM",
   "explanation": "Auth middleware timeout fix directly modifies RBAC token validation
implementation."
  },
  {
   "source_id": "commit_901",
   "target_id": "code_991",
   "label": "changes",
   "confidence": 0.88,
   "system": "LLM",
   "explanation": "Commit message references token expiration validation matching
token_validator responsibilities."
  },
  {
   "source_id": "bug_302",
   "target_id": "bug_101",
   "label": "relates to",
   "confidence": 0.9,
   "system": "LLM",
   "explanation": "Both bugs involve token validation timing and session expiration anomalies."
  },
  {
   "source_id": "feature_999",
   "target_id": "bug_302",
   "label": "refines",
   "confidence": 0.87,
   "system": "LLM",
   "explanation": "Expiry notification feature addresses usability impact of unexpected session
termination."
  },
  {
   "source_id": "feature_044",
   "target_id": "code_401",
   "label": "implements",
   "confidence": 0.95,
   "system": "LLM",
   "explanation": "Middleware source file corresponds directly to RBAC auth middleware
feature."
  },
```

```
  {
    "source_id": "feature_044",
    "target_id": "code_991",
    "label": "implements",
    "confidence": 0.89,
    "system": "LLM",
    "explanation": "Token validator file supports RBAC authentication and validation feature
logic."
  },
  {
    "source_id": "commit_901",
    "target_id": "code_401",
    "label": "modifies",
    "confidence": 0.93,
    "system": "LLM",
    "explanation": "Modifies is more precise than generic changes for direct source edit."
  },
  {
    "source_id": "dev_105",
    "target_id": "bug_302",
    "label": "owns",
    "confidence": 0.91,
    "system": "LLM",
    "explanation": "Owns better reflects active responsibility for bug resolution workflow."
  }
 ]
}
```


Qwen response


```
{
 "new_edges": [
  {
    "source_id": "commit_901",
    "target_id": "feature_044",
    "label": "fixes",
    "confidence": 0.92,
    "system": "LLM",
    "explanation": "Commit fixes the bug caused by feature 044's middleware implementation."
  },
  {
    "source_id": "bug_302",
    "target_id": "code_401",
    "label": "affects",
    "confidence": 0.89,
    "system": "LLM",
    "explanation": "Bug 302 involves middleware.ts file path matching the bug's auth context."
  },
  {
    "source_id": "feature_044",
    "target_id": "code_401",
    "label": "implements",
```

```
    "confidence": 0.87,
    "system": "LLM",
    "explanation": "Feature 044's RBAC middleware directly corresponds to middleware.ts file."
   },
   {
    "source_id": "bug_101",
    "target_id": "feature_999",
    "label": "precedes",
    "confidence": 0.91,
    "system": "LLM",
    "explanation": "Bug 101's token validation issue was resolved before feature 999 was
created."
   },
   {
    "source_id": "bug_101",
    "target_id": "code_991",
    "label": "affects",
    "confidence": 0.88,
    "system": "LLM",
    "explanation": "Bug 101's token validation anomaly directly impacts token_validator.ts file."
   },
   {
    "source_id": "feature_999",
    "target_id": "code_991",
    "label": "implements",
    "confidence": 0.86,
    "system": "LLM",
    "explanation": "Feature 999's session expiry notification requires token_validator.ts
implementation."
   }
  ]
}
```


Prompt V2

The responses of the initial prompt showed promising results. However, the improvement that can be made is about the amount of output that the LLM will generate. A model generating this much possible edges will create an explosion of edges in the graph with a high billing from the output tokens generated. Having many edges in the graph will make the retrieval of graph context slower. Also, the neighbourhood retrieval for graph context will increase for the input, making it harder to understand for the LLM and increases the billing for input tokens. The problem is that parts of the graph might need more edges than other parts, making it implausible to set a limitation. Therefore, I will add that only edges should be created that add value to the graph by really enhancing the semantics or links that were missing between nodes that will not get linked with each other in any other way.

I will also minimize the output also from another angle by only enriching the nodes that are contained in the sliding window. The new created edges will therefore always be related to the new edges. This makes the extra added nodes and edges (neighbourhood retrieval and vector similarity) serve as graph context rather than a sub-sub graph enrichment, creating less output

by having less nodes to relate other nodes to, and having faster answer generation by having less comparisons to make

Another improvement for the prompt is adding the possible relationships labels in the prompt. The schema and graph context nodes might not contain all of the different labels used in the project that the LLM can use. Adding the labels gives more information about what is used often and might bring more awareness on how nodes can relate to each other in the project.

Furthermore, the restriction of using 20 words for an explanation seems a bit too low. An increase of 30 will give more freedom to the LLM to explain their decision.

Therefore, the improved prompt is:

System message

Role
You are a knowledge graph engineer specialised in software traceability. Your task is to analyse existing graph nodes and edges and propose new or improved edges that strengthen semantic links between software artefacts.

Constraints
```
{
 "nodes": {
  "Release": {
   "version": { "type": "string" }
  },
  "Issue": {
   "title": { "type": "string" },
   "type": { "type": "string" },
   "status": { "type": "string"},
   "description": { "type": "string" },
   "created": { "type": "date" },
   "resolved": { "type": "date" }
  },
  "Issue:Feature": {
   "unit of work": { "type": "integer" },
   "business value": { "type": "integer" }
  },
  "Issue:Bug": {
   "detected_date": { "type": "date" },
   "root_cause": { "type": "string" },
   "solved": { "type": "string"}
  },
  "Commit": {
   "title": { "type": "string" },
   "message": { "type": "string" },
   "timestamp": { "type": "string" }
  },
  "Developer": {
   "name": { "type": "string" },
   "email": { "type": "string" }
  },
```

```
  "Code": {
    "file_path": { "type": "string" },
    "deleted": { "type": "string" }
  }
 },
 "edges": {
  "issue_included_in_release": {
    "source_id": "Issue",
    "target_id": "Release",
    "label": "included in"
  },
  "issue_updates_issue": {
    "source_id": "Issue",
    "target_id": "Issue",
    "label": "updates"
  },
  "feature_inherits_issue": {
    "source_id": "Feature",
    "target_id": "Issue",
    "label": "is a"
  },
  "bug_inherits_issue": {
    "source_id": "Bug",
    "target_id": "Issue",
    "label": "is a"
  },
  "feature_relates_to_feature": {
    "source_id": "Feature",
    "target_id": "Feature",
    "label": "relates to"
  },
  "bug_caused_by_feature": {
    "source_id": "Bug",
    "target_id": "Feature",
    "label": "caused by"
  },
  "commit_implements_feature": {
    "source_id": "Commit",
    "target_id": "Feature",
    "label": "implements"
  },
  "commit_solves_bug": {
    "source_id": "Commit",
    "target_id": "Bug",
    "label": "solves"
  },
  "developer_creates_commit": {
    "source_id": "Developer",
    "target_id": "Commit",
    "label": "creates"
  },
  "issue_assigned_to_developer": {
```

```
    "source_id": "Issue",
    "target_id": "Developer",
    "label": "assigned to"
   },
   "commit_changes_code": {
    "source_id": "Commit",
    "target_id": "Code",
    "label": "changes"
   }
  }
}
```

User message

Goal
Given a set of nodes and existing edges from a software traceability knowledge graph, identify missing edges between nodes that are semantically related but not yet connected, and propose synonym edges that replace or supplement vague relationship labels with more precise ones. Return only the new or improved edges that would really enhance the graph in the specified JSON schema with a confidence score higher than 0.85.

Steps
Follow these steps, think through each before writing output:
<step id="1">
Read all nodes inside <graph_content> and build a mental model of what each artefact represents (requirement, test, code module, bug, etc.).
</step>
<step id="2">
Inspect the existing edges from the nodes in the sliding window (which is a key in graph content). Identify: (a) pairs of nodes with NO edge that share an implied semantic relationship, (b) edges whose label is ambiguous or generic (e.g. "related_to", "links") and could be replaced with a domain-specific synonym.
</step>
<step id="3">
For every candidate edge, ask yourself: (a) Does this relationship add traceability value (e.g. satisfies, implements, validates, depends_on, derived_from, refines, or another in <graph edge labels>)? Give priority to the labels in <graph edge labels> when in doubt. (b) Is the direction of the edge correct? (c) Is there enough evidence in the node metadata to justify the edge? (d) Is this edge really needed in the graph? Only proceed with this edge if your confidence is higher than 0.85.
</step>
<step id="4">
Compose the output. For each new or improved edge, fill every field of the JSON schema. Add your confidence of adding that relationship and a short "explanation" string (max 20 words) that explains why that edge was created.
</step>

Example output
```
{
 "new_edges": [
  {
```

```
    "source_id":  " Commit-017",
    "target_id":  " Feature-042",
    "label":     "Implements ",
    "confidence": 0.91,
    "system":      "LLM",
    "explanation":  "Commit 17 implemented the exact API operations specified in feature 42.
Since both have a created date without much time in between, the commit and feature seem to
be created for the same purpose but were not linked."
   },
   {
    "source_id":  "Commit-008",
    "target_id":  "File-042",
    "label":     "Renames",
    "confidence": 0.96,
    "system":      "LLM",
    "explanation":  "Commit 8 renames the file rather than changing it. Since the word renaming
is more descriptive than changing, I used the word ‘renaming’ as synonym label for this edge.”
."
   }
 ]
}

# Graph edge labels
{
   "commit to code": ["modify", "add", "delete"],
   "issue to issue": ["Relates to", "depends upon", "requires", "supercedes", "blocks", "breaks",
"incorporates", "contains", "duplicates", "is a clone of", "blocked", "dependent"],
   "feature to feature": ["Relates to", "depends upon", "requires", "supercedes", "blocks", "breaks",
"incorporates", "contains", "duplicates", "is a clone of", "blocked", "dependent"],
   "bug to bug": ["Relates to", "depends upon", "requires", "supercedes", "blocks", "breaks",
"incorporates", "contains", "duplicates", "is a clone of", "blocked", "dependent"],
   "bug to feature": ["Relates to", "depends upon", "requires", "supercedes", "blocks", "breaks",
"incorporates", "contains", "duplicates", "is a clone of", "blocked", "dependent"],
}

# Graph content
{
 "sliding_window_events": {
   "nodes": {
    "commit_901": {
     "type": "Commit",
     "title": "Fix auth timeout bug",
     "message": "Adjusted JWT verification token expiration buffer to resolve premature session
termination.",
     "timestamp": "2026-05-23T01:15:00Z"
    },
    "code_401": {
     "type": "Code",
     "file_path": "src/auth/middleware.ts",
     "deleted": "false"
    }
   },
```

```
  "edges": {
    "edge_e1": {
      "source_id": "commit_901",
      "target_id": "code_401",
      "label": "changes",
      "system": "git_event_stream",
      "confidence": 100,
      "explanation": "Event log captures commit_901 directly modifying middleware.ts."
    }
  }
 },

 "k_hop_neighbourhood": {
  "nodes": {
    "bug_302": {
      "type": "Issue:Bug",
      "title": "Intermittent 401 on valid user session",
      "status": "in progress",
      "description": "Users are reporting sudden logouts while actively browsing dashboards.",
      "created": "2026-05-22",
      "resolved": null,
      "detected_date": "2026-05-22",
      "root_cause": "Race condition in validation timestamp check.",
      "solved": "no"
    },
    "dev_105": {
      "type": "Developer",
      "name": "Alex Mercer",
      "email": "alex.mercer@company.com"
    },
    "feature_044": {
      "type": "Issue:Feature",
      "title": "Implement RBAC Auth Middleware",
      "status": "done",
      "description": "Establish baseline token validation middleware for role-based access
control.",
      "created": "2026-04-10",
      "resolved": "2026-05-01",
      "unit of work": 5,
      "business value": 80
    }
  },
  "edges": {
    "edge_k1": {
      "source_id": "commit_901",
      "target_id": "bug_302",
      "label": "solves",
      "system": "k_hop_retriever",
      "confidence": 95,
      "explanation": "Hop 1 from sliding window: Commit message explicitly tags or addresses the
context of bug_302."
    },
```

```
    "edge_k2": {
      "source_id": "dev_105",
      "target_id": "commit_901",
      "label": "creates",
      "system": "k_hop_retriever",
      "confidence": 100,
      "explanation": "Hop 1 from sliding window: Author metadata of commit_901 resolves to
dev_105."
    },
    "edge_k3": {
      "source_id": "bug_302",
      "target_id": "feature_044",
      "label": "caused by",
      "system": "k_hop_retriever",
      "confidence": 85,
      "explanation": "Hop 2 from bug_302: Traceability logs show this session regression
originated after feature_044 shipped."
    },
    "edge_k4": {
      "source_id": "bug_302",
      "target_id": "dev_105",
      "label": "assigned to",
      "system": "k_hop_retriever",
      "confidence": 100,
      "explanation": "Hop 2 from bug_302: Issue tracker shows dev_105 owns this ticket."
    }
   }
 },

  "vector_similarity_retrieval": {
   "nodes": {
    "bug_101": {
      "type": "Issue:Bug",
      "title": "Token refresh timing anomaly",
      "status": "done",
      "description": "Legacy authorization system occasionally fails when system clocks are out of
sync.",
      "created": "2025-11-12",
      "resolved": "2025-11-15",
      "detected_date": "2025-11-12",
      "root_cause": "Hardcoded token validation windows with tight tolerance.",
      "solved": "yes"
    },
    "feature_999": {
      "type": "Issue:Feature",
      "title": "User Session Expiry Notification",
      "status": "planned",
      "description": "Provide UI alerts notifying active users that their authentication token is
nearing expiry.",
      "created": "2026-05-19",
      "resolved": null,
      "unit of work": 3,
```

```
      "business value": 40
    },
    "code_991": {
      "type": "Code",
      "file_path": "src/security/token_validator.ts",
      "deleted": "false"
    }
  },
  "edges": {}
 }
}
```


Recap
So, you are a knowledge graph engineer for software traceability. Analyse the nodes and edges in graph content, follow steps 1–4, and return ONLY a JSON object adhering the schema above. Propose missing edges and synonym replacements that really improve semantic precision.

Response GPT


```
{
 "new_edges": [
  {
   "source_id": "commit_901",
   "target_id": "code_401",
   "label": "modify",
   "confidence": 0.97,
   "system": "LLM",
   "explanation": "Commit message adjusts existing middleware logic; file is not deleted, so modify is more precise than changes."
  },
  {
   "source_id": "bug_302",
   "target_id": "bug_101",
   "label": "Relates to",
   "confidence": 0.9,
   "system": "LLM",
   "explanation": "Both bugs concern token timing and session expiry anomalies in authorization logic, retrieved via semantic similarity."
  }
```

```
 ]

}
```

Response Qwen

```
{
   "edges": [
     {
      "source_id": "Issue",
      "target_id": "Release",
      "label": "blocks"
       },
       {
       "node_id": "Issue",
       "target_id": "Release",
        "label": "blocks"
         },
         {
         "source_id": "Issue",
         "target_id": "Release",
         "label": "blocks"
          },
         {
          "source Issue": "Issue",
          "target_id": "Release",
          "label": "blocks"
          },
          {
          "source_id": "Issue",
          "target_id": "Release",
          "blocking_issue": {
                "source_id": "Issue",
                "target_id": "Release",
                "label": "blocks"
                 }
          }
]
}
```

Prompt v3

The improved prompt showed desired and expected results for GPT. However, Qwen had a harder time and showed less quality in its output. Therefore, the next prompt contains more strict writing and more clarity. Also, the output must be forced to be the expected output. Using the role: “assistant” in messages helps with this. Therefore, I will also add the following line to messages to ensure a correct start of the output:

```
{"role": "assistant",  "content": "{\n  \"new_edges\": ["}
```

Also, because Qwen has a smaller memory when it is reading the prompt, not everything will be remembered. Therefore, the prompt will also be more straight to the point and written more

tightly. For example, it is unnecessary for the LLM to know what properties a node can have. I therefore remove the nodes in the schema. Also, the model might get confused by the different JSON notations (expected output, schema, and graph nodes all have JSON format but different structures). I therefore change the schema in the prompt to a Cypher schema notation.

System message

Role

You are a knowledge graph engineer specialised in software traceability. Your task is to analyse existing graph nodes and edges, and propose new or improved edges that strengthen semantic links between software artefacts.

Schema constraints

syntax explained: (source node type)-[edge label]->(target node type)

Allowed relationship patterns (edges) in Cypher schema notation

(:Issue)-[:INCLUDED_IN]->(:Release)

(:Issue)-[:UPDATES]->(:Issue)

(:Issue:Feature)-[:RELATES_TO]->(:Issue:Feature)

(:Issue:Bug)-[:CAUSED_BY]->(:Issue:Feature)

(:Commit)-[:IMPLEMENTS]->(:Issue:Feature)

(:Commit)-[:SOLVES]->(:Issue:Bug)

(:Developer)-[:CREATES]->(:Commit)

(:Issue)-[:ASSIGNED_TO]->(:Developer)

(:Commit)-[:CHANGES]->(:Code)

Predefined edge labels

There are more labels that are defined in the project besides the ones in the schema. Give priority to these labels and the ones in the schema when proposing new edge labels.

Source type	**Target type**	**labels**
Commit	Code	modify, add, delete
Issue	Issue	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
Feature	Feature	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
Bug	Bug	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent

bug	Feature	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent

User message

Goal

Given a set of nodes and existing edges from a software traceability knowledge graph in <graph content>, identify missing edges between nodes that are semantically related but not yet connected, and propose synonym edges that replace or supplement vague relationship labels with more precise ones. Return only the new or improved edges in the specified JSON schema shown in <example output> with a confidence score higher than 0.85.

Steps

Follow these steps strictly—think through each step step-by-step before generating the final JSON output:

Step 1: Build Mental Model

Read all nodes across all retrieval strategies inside <graph_content>. Understand the semantic meaning, type, and metadata of each software artifact (e.g., Commit, Issue:Bug, Issue:Feature, Code, Developer, release).

Step 2: Cross-Strategy Node Comparison

Systematically compare every individual node inside the `sliding_window_events` block in <graph_content> against every single node found in the other retrieval blocks (`k_hop_neighbourhood` and `vector_similarity_retrieval`). For each pair, evaluate if a relationship exists.

Step 3: Identify Candidate and Synonym Edges

During the cross-comparison, identify two types of target links:

(a) Missing Links: Pairs of nodes that currently have NO edge between them but share a clear, implied semantic relationship.

(b) Synonym Replacements: Existing edges with vague or generic labels (e.g., "related_to", "links", "changes") that can be upgraded to a highly precise domain-specific label, prioritizing the predifined edge labels and adhering to the schema constraints.

Step 4: Validate, De-duplicate, and Filter

For every candidate edge found in Step 3, rigorously verify the following criteria:

(a) Sliding Window Anchoring: AT LEAST ONE of the endpoints (`source_id` OR `target_id`) MUST explicitly match a node key located inside the `sliding_window_events.nodes` object. If both nodes belong exclusively to background retrieval strategies, drop the edge immediately.

(b) Meaningful Alignment: Does the chosen label precisely capture the engineering context?

(c) Structural Validity: Is the edge direction accurate based on the schema constraints?

(d) Evidence-Based: Is there sufficient evidence in the titles, descriptions, or timestamps to justify the link?

(e) De-duplication: Ensure this exact relationship does not already exist in the graph data.

(f) Confidence Threshold: Calculate your certainty. Drop the candidate immediately if your confidence score is 0.85 or lower.

Step 5: Construct Final Output

Collect all validated, non-duplicate edges that scored higher than 0.85. Format them exactly into the `new_edges` array using the required JSON output format, ensuring the "explanation" string remains under 50 words.

Output format

```
{
 "new_edges": [
  {
   "source_id": <string value>,
   "target_id": <string value>,
   "label":    <string value>,
   "confidence": <float value>,
   "system":     "LLM",
   "explanation": <string value>
  }
 ]
}
```

Graph content

Content explanation: The first keys are the retrieval strategies. One layer deeper are the nodes and edges retrieved by that strategy. Then come the IDs of the nodes and edges, followed by their properties.

```
{
  "sliding_window_events": {
    "nodes": [
      "commit_901": {
        "type": "Commit",
        "title": "Fix auth timeout bug",
        "message": "Adjusted JWT verification token expiration buffer to resolve premature session termination.",
        "timestamp": "2026-05-23T01:15:00Z"
      },
      "code_401": {
        "type": "Code",
        "file_path": "src/auth/middleware.ts",
        "deleted": "false"
      }
    ],
    "edges": [
      {
        "source_id": "commit_901",
        "target_id": "code_401",
        "label": "changes",
        "system": "git_event_stream",
        "confidence": 100,
        "explanation": "Event log captures commit_901 directly modifying middleware.ts."
      }
    ]
  },

  "k_hop_neighbourhood": {
    "nodes": [{
      "bug_302": {
```

```
    "type": "Issue:Bug",
    "title": "Intermittent 401 on valid user session",
    "status": "in progress",
    "description": "Users are reporting sudden logouts while actively browsing dashboards.",
    "created": "2026-05-22",
    "resolved": null,
    "detected_date": "2026-05-22",
    "root_cause": "Race condition in validation timestamp check.",
    "solved": "no"
   }
  },
  {
   "dev_105": {
    "type": "Developer",
    "name": "Alex Mercer",
    "email": "alex.mercer@company.com"
   }
  },
  {
   "feature_044": {
    "type": "Issue:Feature",
    "title": "Implement RBAC Auth Middleware",
    "status": "done",
    "description": "Establish baseline token validation middleware for role-based access control.",
    "created": "2026-04-10",
    "resolved": "2026-05-01",
    "unit of work": 5,
    "business value": 80
   }
  }
```

```
  ],
  "edges": [
   {
    "source_id": "commit_901",
    "target_id": "bug_302",
    "label": "solves",
    "system": "k_hop_retriever",
    "confidence": 95,
    "explanation": "Hop 1 from sliding window: Commit message explicitly tags or addresses the context of bug_302."
   },
   {
    "source_id": "dev_105",
    "target_id": "commit_901",
    "label": "creates",
    "system": "k_hop_retriever",
    "confidence": 100,
    "explanation": "Hop 1 from sliding window: Author metadata of commit_901 resolves to dev_105."
   },
   {
    "source_id": "bug_302",
    "target_id": "feature_044",
    "label": "caused by",
    "system": "k_hop_retriever",
    "confidence": 85,
    "explanation": "Hop 2 from bug_302: Traceability logs show this session regression originated after feature_044 shipped."
   },
   {
    "source_id": "bug_302",
    "target_id": "dev_105",
```

```
      "label": "assigned to",
      "system": "k_hop_retriever",
      "confidence": 100,
      "explanation": "Hop 2 from bug_302: Issue tracker shows dev_105 owns this ticket."
    }
   ]
 },

 "vector_similarity_retrieval": {
  "nodes": [{
    "bug_101": {
      "type": "Issue:Bug",
      "title": "Token refresh timing anomaly",
      "status": "done",
      "description": "Legacy authorization system occasionally fails when system clocks are out of sync.",
      "created": "2025-11-12",
      "resolved": "2025-11-15",
      "detected_date": "2025-11-12",
      "root_cause": "Hardcoded token validation windows with tight tolerance.",
      "solved": "yes"
    }
  },
  {
    "feature_999": {
      "type": "Issue:Feature",
      "title": "User Session Expiry Notification",
      "status": "planned",
      "description": "Provide UI alerts notifying active users that their authentication token is nearing expiry.",
      "created": "2026-05-19",
      "resolved": null,
```

```
      "unit of work": 3,
      "business value": 40
    }
   },
   {
    "code_991": {
      "type": "Code",
      "file_path": "src/security/token_validator.ts",
      "deleted": "false"
    }
   }
   ],
   "edges": []
 }
}
```


Recap

You are a knowledge graph engineer for software traceability. Analyse the nodes and edges in graph content, follow steps 1–5, propose missing edges and synonym replacements that improve semantic precision, and return ONLY JSON output using the <output format>.

Response GPT


```
{
 "new_edges": [
  {
    "source_id": "commit_901",
    "target_id": "code_401",
    "label": "modify",
    "confidence": 0.97,
    "system": "LLM",
    "explanation": "Commit_901 adjusts logic in src/auth/middleware.ts, indicating the file is modified rather than added or deleted."
  }
```

```
 ]
}
```


Response Qwen


```
{
 "new_edges": [
  {
   "source_id": "commit_901",
   "target_id": "code_991",
   "label": "modifies",
   "confidence": 0.92,
   "system": "LLM",
   "explanation": "Commit fixes JWT token expiration, directly impacting the token_validator.ts file."
  },
  {
   "source_id": "bug_302",
   "target_id": "feature_999",
   "label": "supersedes",
   "confidence": 0.88,
   "system": "LLM",
   "explanation": "Bug describes legacy token issues; feature proposes a proactive notification solution to prevent them."
  },
  {
   "source_id": "bug_302",
   "target_id": "bug_101",
   "label": "is a clone of",
   "confidence": 0.9,
   "system": "LLM",
   "explanation": "Both bugs describe token expiration/session timeout issues with similar root causes."
  }
```

]

}

Prompt v4

The results were almost correct. The only thing that needs to be improved is that only the edges are enriched that were coming from the sliding window retrieval. This reduces the amount of edges created (since LLMs will create output most of the time, no matter how good or bad the query is). However, the LLM might still enrich the graph with edges between other nodes, since the models are both reasoning models and the temperature might not be completely set to 0 in the background to make the LLM able to reason. Emphasizing the importance of enriching the nodes from the sliding window is therefore the only thing I can add to increase the chance of giving desired results by the LLMs.

System message

Role

You are a knowledge graph engineer specialised in software traceability. Your task is to analyse existing graph nodes and edges, and propose new or improved edges that strengthen semantic links between software artefacts.

Schema constraints

syntax explained: (source node type)-[edge label]->(target node type)

Allowed relationship patterns (edges) in Cypher schema notation

(:Issue)-[:INCLUDED_IN]->(:Release)

(:Issue)-[:UPDATES]->(:Issue)

(:Issue:Feature)-[:RELATES_TO]->(:Issue:Feature)

(:Issue:Bug)-[:CAUSED_BY]->(:Issue:Feature)

(:Commit)-[:IMPLEMENTS]->(:Issue:Feature)

(:Commit)-[:SOLVES]->(:Issue:Bug)

(:Developer)-[:CREATES]->(:Commit)

(:Issue)-[:ASSIGNED_TO]->(:Developer)

(:Commit)-[:CHANGES]->(:Code)

Predefined edge labels

There are more labels that are defined in the project besides the ones in the schema. Give priority to these labels and the ones in the schema when proposing new edge labels.

Source type	**Target type**	**labels**

Commit	Code	modify, add, delete
Issue	Issue	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
Feature	Feature	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
Bug	Bug	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
bug	Feature	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent

User message

Goal

Given a set of nodes and existing edges from a software traceability knowledge graph in <graph content>, identify missing edges between nodes that are semantically related but not yet connected, and propose synonym edges that replace or supplement vague relationship labels with more precise ones. Return only the new or improved edges in the specified JSON schema shown in <example output> with a confidence score higher than 0.85.

Steps

Follow these steps strictly—think through each step step-by-step before generating the final JSON output:

Step 1: Build Mental Model

Read all nodes across all retrieval strategies inside <graph_content>. Understand the semantic meaning, type, and metadata of each software artifact (e.g., Commit, Issue:Bug, Issue:Feature, Code, Developer, release).

Step 2: Anchored Cross-Comparison Loop

Begin your analysis loops exclusively from the current context. Take the first node from `sliding_window_events.nodes` as the Anchor Node. Compare this Anchor Node systematically against every node in `k_hop_neighbourhood.nodes` and `vector_similarity_retrieval.nodes`. Once completed, repeat the loop using the next node from `sliding_window_events.nodes`.

Step 3: Identify Candidate and Synonym Edges

During the cross-comparison, identify two types of target links:

(a) Missing Links: Pairs of nodes that currently have NO edge between them but share a clear, implied semantic relationship.

(b) Synonym Replacements: Existing edges with vague or generic labels (e.g., "related_to", "links", "changes") that can be upgraded to a highly precise domain-specific label, prioritizing the predifined edge labels and adhering to the schema constraints.

Step 4: Validate, De-duplicate, and Filter

For every candidate edge found in Step 3, rigorously verify the following criteria:

(a) Sliding Window Anchoring: AT LEAST ONE of the endpoints (`source_id` OR `target_id`) MUST explicitly match a node key located inside the `sliding_window_events.nodes` object. If none of the nodes belong to the sliding window strategy, drop the edge immediately.

(b) Meaningful Alignment: Does the chosen label precisely capture the engineering context?

(c) Structural Validity: Is the edge direction accurate based on the schema constraints?

(d) Evidence-Based: Is there sufficient evidence in the titles, descriptions, or timestamps to justify the link?

(e) De-duplication: Ensure this exact relationship does not already exist in the graph data.

(f) Confidence Threshold: Calculate your certainty. Drop the candidate immediately if your confidence score is 0.85 or lower.

Step 5: Construct Final Output

Collect all validated, non-duplicate edges that scored higher than 0.85. Format them exactly into the `new_edges` array using the required JSON output format, ensuring the "explanation" string remains under 50 words.

Output format

```
{
 "new_edges": [
  {
   "source_id":  <string value>,
   "target_id":  <string value>,
   "label":     <string value>,
   "confidence": <float value>,
   "system":      "LLM",
   "explanation":  <string value>
```

```
  }
 ]
}
```


Graph content

Content explanation: The first keys are the retrieval strategies. One layer deeper are the nodes and edges retrieved by that strategy. Then come the IDs of the nodes and edges, followed by their properties.


```
{
 "sliding_window_events": {
  "nodes": [
   "commit_901": {
    "type": "Commit",
    "title": "Fix auth timeout bug",
    "message": "Adjusted JWT verification token expiration buffer to resolve premature session termination.",
    "timestamp": "2026-05-23T01:15:00Z"
   },
   "code_401": {
    "type": "Code",
    "file_path": "src/auth/middleware.ts",
    "deleted": "false"
   }
  ],
  "edges": [
   {
    "source_id": "commit_901",
    "target_id": "code_401",
    "label": "changes",
    "system": "git_event_stream",
    "confidence": 100,
    "explanation": "Event log captures commit_901 directly modifying middleware.ts."
```

```
   }
  ]
},

"k_hop_neighbourhood": {
  "nodes": [{
    "bug_302": {
      "type": "Issue:Bug",
      "title": "Intermittent 401 on valid user session",
      "status": "in progress",
      "description": "Users are reporting sudden logouts while actively browsing dashboards.",
      "created": "2026-05-22",
      "resolved": null,
      "detected_date": "2026-05-22",
      "root_cause": "Race condition in validation timestamp check.",
      "solved": "no"
    }
  },
  {
    "dev_105": {
      "type": "Developer",
      "name": "Alex Mercer",
      "email": "alex.mercer@company.com"
    }
  },
  {
    "feature_044": {
      "type": "Issue:Feature",
      "title": "Implement RBAC Auth Middleware",
      "status": "done",
```

```
      "description": "Establish baseline token validation middleware for role-based access control.",
      "created": "2026-04-10",
      "resolved": "2026-05-01",
      "unit of work": 5,
      "business value": 80
    }
   }
   ],
   "edges": [
    {
     "source_id": "commit_901",
     "target_id": "bug_302",
     "label": "solves",
     "system": "k_hop_retriever",
     "confidence": 95,
     "explanation": "Hop 1 from sliding window: Commit message explicitly tags or addresses the context of bug_302."
    },
    {
     "source_id": "dev_105",
     "target_id": "commit_901",
     "label": "creates",
     "system": "k_hop_retriever",
     "confidence": 100,
     "explanation": "Hop 1 from sliding window: Author metadata of commit_901 resolves to dev_105."
    },
    {
     "source_id": "bug_302",
     "target_id": "feature_044",
     "label": "caused by",
```

```
      "system": "k_hop_retriever",
      "confidence": 85,
      "explanation": "Hop 2 from bug_302: Traceability logs show this session regression originated after feature_044 shipped."
    },
    {
      "source_id": "bug_302",
      "target_id": "dev_105",
      "label": "assigned to",
      "system": "k_hop_retriever",
      "confidence": 100,
      "explanation": "Hop 2 from bug_302: Issue tracker shows dev_105 owns this ticket."
    }
  ]
 },


 "vector_similarity_retrieval": {
  "nodes": [{
   "bug_101": {
     "type": "Issue:Bug",
     "title": "Token refresh timing anomaly",
     "status": "done",
     "description": "Legacy authorization system occasionally fails when system clocks are out of sync.",
     "created": "2025-11-12",
     "resolved": "2025-11-15",
     "detected_date": "2025-11-12",
     "root_cause": "Hardcoded token validation windows with tight tolerance.",
     "solved": "yes"
    }
  },
  {
```

```
    "feature_999": {
      "type": "Issue:Feature",
      "title": "User Session Expiry Notification",
      "status": "planned",
      "description": "Provide UI alerts notifying active users that their authentication token is nearing expiry.",
      "created": "2026-05-19",
      "resolved": null,
      "unit of work": 3,
      "business value": 40
    }
  },
  {
    "code_991": {
      "type": "Code",
      "file_path": "src/security/token_validator.ts",
      "deleted": "false"
    }
  }
  ],
  "edges": []
 }
}
```


Recap

You are a knowledge graph engineer for software traceability. Analyse the nodes and edges in graph content, follow steps 1–5, propose missing edges and synonym replacements that improve semantic precision, and return ONLY JSON output using the <output format>.

Response GPT


```
{
 "new_edges": [
  {
```

```
    "source_id": "commit_901",
    "target_id": "code_401",
    "label": "modify",
    "confidence": 0.97,
    "system": "LLM",
    "explanation": "Refines generic 'changes' relation to precise Commit\u2192Code action using the predefined 'modify' label for edits to existing files."
   }
 ]
}
```


Response Qwen


```
{
 "new_edges": [
  {
    "source_id": "commit_901",
    "target_id": "bug_101",
    "label": "fixes",
    "confidence": 0.92,
    "system": "LLM",
    "explanation": "Commit message 'Fix auth timeout bug' and description of premature session termination semantically match the root cause of bug_101 regarding token refresh timing."
  },
  {
    "source_id": "commit_901",
    "target_id": "feature_999",
    "label": "implements",
    "confidence": 0.88,
    "system": "LLM",
    "explanation": "Commit adjusts JWT expiration buffer, which directly addresses the functionality described in feature_999 for notifying users of token expiry."
  },
  {
```

"source_id": "code_401",

"target_id": "bug_101",

"label": "contains",

"confidence": 0.86,

"system": "LLM",

"explanation": "The file path 'src/auth/middleware.ts' is the logical location for the token validation logic identified as the root cause of bug_101."

}

]

}

Inserting the prompt in the pipeline

Now that the prompt responses with the desired result, I need to test if the prompt also works in production. This gives the following result for Qwen:

response: {

"new_edges": [

{

"source_id": "Jason van Zyl",

"target_id": "412a00a07db2b91cfd39d80a18f7c08224a6515c",

"label": "CREATES",

"confidence": 0.95,

"system": "LLM",

"explanation": "Commit message explicitly states 'Submitted by: Jason van Zyl', establishing direct authorship."

},

{

"source_id": "Jason van Zyl",

"target_id": "1a3dd53ada1bd479590459bf7d0d24887f202afa",

"label": "CREATES",

"confidence": 0.95,

"system": "LLM",

"explanation": "Commit message explicitly states 'Submitted by: Jason van Zyl', establishing direct authorship."

```
  },
  {
    "source_id": "Jason van Zyl",
    "target_id": "velocity.log",
    "label": "MODIFIES",
    "confidence": 0.92,
    "system": "LLM",
    "explanation": "Commit 412a00a07db2b91cfd39d80a18f7c08224a6515c, created by Jason van Zyl, deletes velocity.log."
  },
  {
    "source_id": "Jason van Zyl",
    "target_id": "API.html",
    "label": "MODIFIES",
    "confidence": 0.92,
    "system": "LLM",
    "explanation": "Commit 1a3dd53ada1bd479590459bf7d0d24887f202afa, created by Jason van Zyl, adds API.html."
  },
  {
    "source_id": "Jason van Zyl",
    "target_id": "datasheet.html",
    "label": "MODIFIES",
    "confidence": 0.92,
    "system": "LLM",
    "explanation": "Commit 1a3dd53ada1bd479590459bf7d0d24887f202afa, created by Jason van Zyl, adds datasheet.html."
  },
  {
    "source_id": "Jason van Zyl",
    "target_id": "LICENSE",
    "label": "MODIFIES",
```

```
    "confidence": 0.92,
    "system": "LLM",
    "explanation": "Commit 412a00a07db2b91cfd39d80a18f7c08224a6515c, created by Jason van Zyl, adds LICENSE."
  },
  {
    "source_id": "Jason van Zyl",
    "target_id": ".cvsignore",
    "label": "MODIFIES",
    "confidence": 0.92,
    "system": "LLM",
    "explanation": "Commit 412a00a07db2b91cfd39d80a18f7c08224a6515c, created by Jason van Zyl, adds .cvsignore."
  },
  {
    "source_id": "412a00a07db2b91cfd39d80a18f7c08224a6515c",
    "target_id": "Apache License 2.0",
    "label": "DESCRIBES",
    "confidence": 0.90,
    "system": "LLM",
    "explanation": "The LICENSE file content in the commit contains the text 'Obtained from: https://svn.apache.org/repos/asf/lucene/java/trunk@149577 13f79535-47bb-0310-9956-ffa450edef68', identifying the source of the Apache License 2.0."
  },
  {
    "source_id": "412a00a07db2b91cfd39d80a18f7c08224a6515c",
    "target_id": "Jason van Zyl",
    "label": "DESCRIBES",
    "confidence": 0.90,
    "explanation": "The commit message includes 'Obtained from: https://svn.apache.org/repos/asf/lucene/java/trunk@149577 13f79535-47bb-0310-9956-ffa450edef68', identifying the source of the Apache License 2.0."
  }
```

```
 ]
}
```


The second response:


```
response: {
 "new_edges": [
  {
    "source_type": "Commit",
    "target_type": "Issue",
    "label": "RESOLVES"
  },
  {
    "allowed_source_type": "Commit",
    "allowed_target_type": "Code",
    "label": "MODIFIES"
  },
  {
    "source_type": "Issue",
    "target_type": "Release",
    "label": "RESOLVES"
  },
  {
    "source_type": "Commit",
    "target_type": "Commit",
    "label": "MERGES"
 }
]
}
```


These two results, which came one after the other, show that the first response was good, but the second cannot be used, also the responses after the second were of the same quality of the second response. An improvement has to be made to make Qwen work in the continuous environment. Fortunately, GPT responses are the expected result.

Prompt v5

When using the prompt in the pipeline, the Qwen responses show that I have to adjust the quality of the prompt, as well as some parameters. Therefore, I give the CPU time to breath. The CPU is currently operating at a usage of 1000%, meaning 10 cores, out of 16, are constantly in use. Building a sleep in the program would give them a bit of rest. Also, Qwen does not seem to adhere to the output structure. To solve this, I put the output structure in the system message at the top, to make it aware of what the output should look like. Another improvement would be the suggestion to return an empty list for the edge case of zero new edges.

Since the GPT model returns better and desired results, it is more important to adjust the Qwen prompt to see when the desired result is achieved. Also, requesting GPT all the time consumes tokens and thus money. To achieve the goal of a good response from Qwen while keeping the costs low, the next prompt is only performed on the Qwen model. The final prompt will be tested with GPT.

System message

Role

You are a knowledge graph engineer specialised in software traceability. Your task is to analyse existing graph nodes and edges, and propose new or improved edges that strengthen semantic links between software artefacts.

Critical output rule

{

"new_edges": [

{

"source_id": <string value>,

"target_id": <string value>,

"label": <string value>,

"confidence": <float value>,

"system": "LLM",

"explanation": <string value>

}

]

}

Schema constraints

syntax explained: (source node type)-[edge label]->(target node type)

Allowed relationship patterns (edges) in Cypher schema notation

(:Issue)-[:INCLUDED_IN]->(:Release)

(:Issue)-[:UPDATES]->(:Issue)

(:Issue:Feature)-[:RELATES_TO]->(:Issue:Feature)

(:Issue:Bug)-[:CAUSED_BY]->(:Issue:Feature)

(:Commit)-[:IMPLEMENTS]->(:Issue:Feature)

(:Commit)-[:SOLVES]->(:Issue:Bug)

(:Developer)-[:CREATES]->(:Commit)

(:Issue)-[:ASSIGNED_TO]->(:Developer)

(:Commit)-[:CHANGES]->(:Code)

Predefined edge labels

There are more labels that are defined in the project besides the ones in the schema. Give priority to these labels and the ones in the schema when proposing new edge labels.

Source type	**Target type**	**labels**
Commit	Code	modify, add, delete
Issue	Issue	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
Feature	Feature	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
Bug	Bug	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
bug	Feature	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent

User message

Goal

Given a set of nodes and existing edges from a software traceability knowledge graph in <graph content>, identify missing edges between nodes that are semantically related but not yet connected, and propose synonym edges that replace or supplement vague relationship labels with more precise ones. Respond with ONLY a valid JSON, having only the key "new_edges", containing a list with dicts that contain the keys: "source_id", "target_id", "label", "confidence", "system", and "explanation", and with a confidence score higher than 0.85.

Steps

Follow these steps strictly—think through each step step-by-step before generating the final JSON output:

Step 1: Build Mental Model

Read all nodes across all retrieval strategies inside <graph_content>. Understand the semantic meaning, type, and metadata of each software artifact (e.g., Commit, Issue:Bug, Issue:Feature, Code, Developer, release).

Step 2: Anchored Cross-Comparison Loop

Begin your analysis loops exclusively from the current context. Take the first node from `sliding_window_events.nodes` as the Anchor Node. Compare this Anchor Node systematically against every node in `k_hop_neighbourhood.nodes` and `vector_similarity_retrieval.nodes`. Once completed, repeat the loop using the next node from `sliding_window_events.nodes`.

Step 3: Identify Candidate and Synonym Edges

During the cross-comparison, identify two types of target links:

(a) Missing Links: Pairs of nodes that currently have NO edge between them but share a clear, implied semantic relationship.

(b) Synonym Replacements: Existing edges with vague or generic labels (e.g., "related_to", "links", "changes") that can be upgraded to a highly precise domain-specific label, prioritizing the predifined edge labels and adhering to the schema constraints.

Step 4: Validate, De-duplicate, and Filter

For every candidate edge found in Step 3, rigorously verify the following criteria:

(a) Sliding Window Anchoring: AT LEAST ONE of the endpoints (`source_id` OR `target_id`) MUST explicitly match a node key located inside the `sliding_window_events.nodes` object. If none of the nodes belong to the sliding window strategy, drop the edge immediately.

(b) Meaningful Alignment: Does the chosen label precisely capture the engineering context?

(c) Structural Validity: Is the edge direction accurate based on the schema constraints?

(d) Evidence-Based: Is there sufficient evidence in the titles, descriptions, or timestamps to justify the link?

(e) De-duplication: Ensure this exact relationship does not already exist in the graph data.

(f) Confidence Threshold: Calculate your certainty. Drop the candidate immediately if your confidence score is 0.85 or lower.

Step 5: Construct Final Output

Collect all validated, non-duplicate edges that scored higher than 0.85. Format them exactly into the `new_edges` array using the required JSON output rule structure, ensuring the "explanation" string remains under 50 words.

Graph content

Content explanation: The first keys are the retrieval strategies. One layer deeper are the nodes and edges retrieved by that strategy. Then come the IDs of the nodes and edges, followed by their properties.

{graph_content}

Recap

You are a knowledge graph engineer for software traceability. Analyse the nodes and edges in graph content, follow steps 1–5, propose missing edges and synonym replacements that improve semantic precision. Respond with ONLY a valid JSON, having only the

key "new_edges", containing a list with dicts that contain the keys: "source_id", "target_id", "label", "confidence", "system", and "explanation", and with a confidence score higher than 0.85. If no valid edges are found, respond with {{"new_edges": []}}.

Response Qwen


```
{
  "new_edges": [
    {
      "source_id": "COMMIT-1234",
      "target_id": "CODE-5678",
      "label": "MODIFIES",
      "confidence": 0.95,
      "system": "LLM",
      "explanation": "This edge captures the direct relationship between a commit and the specific code changes it introduces, strengthening traceability for code review and impact analysis."
    },
    {
      "source_id": "ISSUE-9012",
      "target_id": "ISSUE-3456",
      "label": "BLOCKS",
      "confidence": 0.92,
```

```
    "system": "LLM",
    "label": "BLOCKS",
    "explanation": "This edge indicates that Issue-9012 must be resolved before Issue-3456 can be addressed, establishing a clear dependency chain for project planning."
   },
   {
    "source_id": "FEATURE-7890",
    "target_id": "BUG-1122",
    "label": "INCORPORATES",
    "confidence": 0.88,
    "system": "LLM",
    "explanation": "This edge shows that Feature-7890 includes the resolution of Bug-1122, linking feature implementation to bug fixes."
   }
  ]
}
```


Second response:


```
{
  "new_edges": [
   {
    "source_id": "COMMIT-1024",
    "target_id": "ISSUE-456",
    "label": "SOLVES",
    "confidence": 0.95,
    "system": "LLM",
    "explanation": "The commit message explicitly states it fixes a regression in the authentication module, which matches the description of ISSUE-456 (a bug reported in the auth module). The commit hash is present in the issue's 'fixed by' field."
   },
   {
    "source_id": "ISSUE-789",
    "target_id": "ISSUE-102",
```

"label": "DEPENDS_UPON",

"confidence": 0.88,

"system": "LLM",

"resolves": "ISSUE-789 describes a feature request for a new dashboard widget, which requires the underlying data model changes described in ISSUE-102. The dependency is inferred from the feature's technical requirements."

},

{

"source_id": "COMMIT-2048",

"target_id": "CODE-3321",

"label": "MODIFY",

"diff": "The commit modifies the user profile service, which directly corresponds to CODE-3321 (the user profile service implementation file). The diff analysis confirms the changes are in the correct file."

},

{

"source

Prompt v6

The previous version showed improvements, but needs a helper function to make the output of the Json format. Also, not each enriched edge can be used because of the keys not being complete or have the same type (e.g. “diff” or “response” used instead of explanation). Another observation is that Qwen creates its own IDs. The dataset uses different IDs and cannot be linked to nodes in the graph, making insertion of the enrichment impossible. Lastly, the prompt should have an empty case example to let the LLM know what to do when there are no enrichments. Most of this can be achieved by adjusting the prompt. Nevertheless, the structure of the response seems much better and could actually be used.

System message

Role

You are a knowledge graph engineer specialised in software traceability. Your task is to analyse existing graph nodes and edges, and propose new or improved edges that strengthen semantic links between software artefacts.

Schema type constraints

syntax explained: (source node type)-[edge label]->(target node type)

Allowed relationship patterns (edges) in Cypher schema notation

(:Issue)-[:INCLUDED_IN]->(:Release)

(:Issue)-[:UPDATES]->(:Issue)

(:Issue:Feature)-[:RELATES_TO]->(:Issue:Feature)

(:Issue:Bug)-[:CAUSED_BY]->(:Issue:Feature)

(:Commit)-[:IMPLEMENTS]->(:Issue:Feature)

(:Commit)-[:SOLVES]->(:Issue:Bug)

(:Developer)-[:CREATES]->(:Commit)

(:Issue)-[:ASSIGNED_TO]->(:Developer)

(:Commit)-[:CHANGES]->(:Code)

Predefined edge labels

There are more labels that are defined in the project besides the ones in the schema. Give priority to these labels and the ones in the schema when proposing new edge labels.

Source type	**Target type**	**labels**
Commit	Code	modify, add, delete
Issue	Issue	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
Feature	Feature	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
Bug	Bug	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent
bug	Feature	relates to, depends upon, requires, supersedes, blocks, breaks, incorporates, contains, duplicates, is a clone of, blocked, dependent

User message

Goal

Given a set of nodes and existing edges from a software traceability knowledge graph in <graph content>, identify missing edges between nodes that are semantically related but not yet connected, and propose synonym edges that replace or supplement vague relationship labels with more precise ones. Respond with ONLY a valid JSON, having only the key "new_edges", containing a list with dicts that contain the keys: "source_id", "target_id", "label", "confidence", "system", and "explanation", and with a confidence score higher than 0.85.

Graph content

Content explanation: The first keys are the retrieval strategies. One layer deeper are the nodes and edges retrieved by that strategy. Then come the IDs of the nodes and edges, followed by their properties.

{graph_content}

Steps

Follow these steps, strictly—think through each step before generating the final JSON output:

Step 1:

Read all nodes across all retrieval strategies inside <graph_content>. Understand the semantic meaning, type, and metadata of each software artifact (e.g., Commit, Issue:Bug, Issue:Feature, Code, Developer, release).

Step 2:

Begin your analysis loops exclusively from the current context. Take the first node from `sliding_window_events.nodes` as the Anchor Node. Compare this Anchor Node systematically against every node in `k_hop_neighbourhood.nodes` and `vector_similarity_retrieval.nodes`. Once completed, repeat the loop using the next node from `sliding_window_events.nodes`.

Step 3:

During the cross-comparison, identify two types of target links:

(a) Missing Links: Pairs of nodes that currently have NO edge between them but share a clear, implied semantic relationship.

(b) Synonym Replacements: Existing edges with vague or generic labels (e.g., "related_to", "links", "changes") that can be upgraded to a highly precise domain-specific label, prioritizing the predifined edge labels and adhering to the schema constraints.

Step 4:

For every candidate edge found in Step 3, rigorously verify the following criteria:

(a) Sliding Window Anchoring: AT LEAST ONE of the endpoints (`source_id` OR `target_id`) MUST explicitly match a node key located inside the `sliding_window_events.nodes` object. If none of the nodes belong to the sliding window strategy, drop the edge immediately.

(b) Meaningful Alignment: Does the chosen label precisely capture the engineering context?

(c) Structural Validity: Is the edge direction accurate based on the schema constraints?

(d) Evidence-Based: Is there sufficient evidence in the titles, descriptions, or timestamps to justify the link?

(e) De-duplication: Ensure this exact relationship does not already exist in the graph data.

(f) Confidence Threshold: Calculate your certainty. Drop the candidate immediately if your confidence score is 0.85 or lower.

Step 5: Construct Final Output

Collect all validated and non-duplicate edges that scored higher than 0.85. Format them exactly into the `new_edges` array using the required JSON output rule structure, ensuring the "explanation" string remains under 50 words.

Critical output rule

{{

"new_edges": [

{{

"source_id": <exact value of id key from graph_content>,

"target_id": <exact value of id key from graph_content>,

"label": <string value>,

"confidence": <float value>,

"system": "LLM",

"explanation": <string value>

}}

]

}}

VALID NODE IDs — STRICT RULES

- You MUST only use the exact keys present in the provided <graph_content>.

- DO NOT use generic placeholders, example IDs, or common shorthand labels like "ISSUE-101", "ISSUE-102", or "TASK-1".

- If you use an ID that does not exist exactly as a key inside <graph_content>, the output is completely invalid.

Recap

You are a knowledge graph engineer for software traceability. Analyse the nodes and edges in graph content, follow steps 1–5, and propose missing edges and synonym replacements that improve the graph by being highly selective and do not over-connect the nodes. Respond with

ONLY a valid JSON, having only the key "new_edges", containing a list with dicts that contain the keys: "source_id", "target_id", "label", "confidence", "system", and "explanation", and with a confidence score higher than 0.85. If no valid edges are found or can be established, respond with {{"new_edges": []}}.

Qwen response

First run


```
{
  "new_edges": [
    {
      "source_id": "COMMIT-1234",
      "target_id": "ISSUE-5678",
      "label": "SOLVES",
      "confidence": 0.9,
      "system": "LLM",
      "explanation": "The commit message explicitly states it fixes a regression in the login module that was reported as ISSUE-5678."
    },
    {
      "source_id": "ISSUE-1234",
      "target_id": "FEATURE-9012",
      "label": "BLOCKS",
      "confidence": 0.8,
      "system": "impact_analysis_engine",
      "explanation": "The feature implementation requires changes to the authentication module which is currently blocked by ISSUE-1234."
    },
    {
      "
```


Second run


```
{
  "new_edges": [
    {
      "source_id": "COMMIT-1234",
```

"target_id": "ISSUE-5678",

"label": "SOLVES",

"confidence": 0.9,

"system": "LLM",

"explanation": "The commit message explicitly states it fixes a regression in the login module that was reported as ISSUE-5678."

},

{

"source_id": "ISSUE-1234",

"target_id": "FEATURE-9012",

"label": "BLOCKS",

"confidence": 0.8,

"system": "impact_analysis_engine",

"explanation": "The feature implementation requires changes to the authentication module which is currently blocked by ISSUE-1234."

},

{

"

Final verdict

The final prompt shows a correct structure, but wrong and hallucinated content. Those IDs are leading for that observation, making me consider that the LLM falls back to its training data and produces output from that. Therefore, more in depth parameter tuning is required.

This response can have different causes. The cache, token amount, and number of output tokens can all be part of the problem. Using logs, I found the following:

Prompt input tokens: 26341

Output tokens: 444

Stop reason: length

eval_count: 195

These logs show that the reason for ending is ‘length’, meaning that the prompt is too big. The number of output tokens is low, 444, which means that output tokens cannot be the cause. The cache score can be observed by logging similarity score from the Qwen model. This score shows the number 195, which means that the prompt is evaluated by the model, whereas 0 shows that the prompt did not evaluate the model and got everything from cache. The results require to reduce the graph content or increase the number of input tokens.

After those adjustments were made. The LLM gave valid and correct results.

Parameters

```
        model=QWEN_MODEL_NAME,
        messages=messages_object(graph_content),
        format="json",
        keep_alive=0,
        options={
          "temperature": 0.1,
          "num_ctx": 36864,
          "num_predict": 2048,
          "num_batch": 512,
          "seed": random.randint(1, 9999999),
          "num_thread": 6,
          "stop": ["]\n}"],
        },
```

The message:

```
[
    {"role": "system",    "content": load_system_prompt(random.randint(0,999999))},
    {"role": "user",      "content": user_prompt},
    {"role": "assistant", "content": "{\n  \"new_edges\": ["}
  ]
```

Everything can be found in the replication package.